

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Лабораторная работа №3

по дисциплине «Проектирование защищенных интеллектуальных
информационных систем»

на тему «Анализ безопасности кода.

Часть 1. Анализ безопасности кода разработанного приложения»

Выполнил студент гр. 321701

Ковальчук В. Н.
Самович В. М.

Проверил

Сальников Д. А.

Минск 2026

Цель: научиться анализировать безопасность кода приложения и поведение конфиденциальных данных в оперативной памяти процесса.

Задание:

а) Разработать приложение с интерактивным интерфейсом, позволяющее создавать, обновлять и удалять в оперативной памяти конфиденциальные (зашифрованные) и неконфиденциальные данные.

б) Выполнить анализ безопасности приложения с точки зрения кода.

в) Выполнить анализ оперативной памяти и процессорного времени.

г) Создать дампы памяти после ввода, обновления и удаления данных и проанализировать дампы.

д) Оформить отчёт.

Постановка задачи. Для выполнения работы приложение **SecureApp** из лабораторной работы №2.1 дополнено разделом «In-memory data». Его записи хранятся только в памяти backend-сервера и не пишутся ни в базу данных, ни в файл.

Краткое описание разработанного приложения

В backend добавлен сервис **InMemoryRecordStore**, который хранит записи в двух **ConcurrentDictionary**: отдельно конфиденциальные и неконфиденциальные. Поверх него работают два контроллера с маршрутами `/api/memory/confidential` и `/api/memory/public` (создание, чтение, обновление, удаление, поиск). На фронтенде для них добавлены две вкладки с тем же интерфейсом, что и у постоянных данных.

Конфиденциальные поля шифруются AES-256-GCM (класс **ConfidentialCrypto** из лабораторной работы №2.1) в момент создания или обновления записи, поэтому в словаре лежат только зашифрованные байты (листинг 1).

```
1 public ConfidentialEntry CreateConfidential(int ownerId, string title, string content)
2 {
3     var id = Interlocked.Increment(ref _nextConfidentialId);
4     var now = DateTime.UtcNow;
5     var entry = new ConfidentialEntry(id, ownerId, Encrypt(title), Encrypt(content),
6     ↪ now, now);
7     _confidential[id] = entry;
8     return entry;
9 }
```

Листинг 1 – Создание конфиденциальной записи: шифрование до помещения в словарь

Анализ защищённости приложения с точки зрения кода

– **Невалидные данные.** Размер и обязательность полей заданы атрибутами `DataAnnotations` в DTO (`Title` до 200 символов, `Content` до 10 000). Неверный тип поля, некорректный JSON, отсутствующее поле, пустое тело и слишком длинное значение приводят к ответу 400 без раскрытия внутренних деталей; проверка этих случаев приведена в части 2 лабораторной работы.

– **Конфиденциальные данные.** Данные шифруются сразу при вводе, ключ берётся из конфигурации, а не из кода. Промежуточные байтовые буферы в `ConfidentialCrypto` очищаются через `Array.Clear`. Строки `string` в .NET неизменяемы, поэтому очистить открытый текст, полученный из запроса, код приложения не может.

– **Параллельные запросы.** Используются `ConcurrentDictionary` и `Interlocked.Increment`, поэтому два запроса не получают один идентификатор.

Анализ оперативной памяти и процессорного времени

Показатели процесса измерялись утилитой `dotnet-counters` (исходные данные – `lab/code/counters_load.csv`) в состоянии покоя и под нагрузкой в 900 запросов (300 циклов создание → обновление → удаление) за около 10 секунд (таблица 1).

Показатель	Покой	Нагрузка (максимум)
Рабочий набор памяти, МБ	260	293
Время CPU в секунду (user), с	0,001–0,13	1,09
Рабочих элементов пула потоков в секунду	0	877

Таблица 1 – Память и процессорное время приложения

Прирост памяти за 900 операций составил около 33 МБ, после окончания нагрузки загрузка CPU вернулась к уровню покоя. Признаков утечки памяти нет.

Анализ дампов оперативной памяти

Дампы снимаются скриптом `dump_analysis.sh` (листинг 2) командой `dotnet-dump collect` после создания, обновления и удаления двух записей. В каждой записи есть уникальный маркер: `CONFIDENTIAL-MARKER-1` (после обновления – -2) для конфиденциальной и `PUBLIC-MARKER-1` (после обновления – -2) для открытой. Дампы просматриваются в VS Code в расширении Hex Editor, где маркер ищется как обычная строка.

```
1  #!/usr/bin/env bash
2  # Usage: ./dump_analysis.sh (the SecureApp.Api process is found automatically)
3  # Creates records, updates, deletes them and takes a memory dump after each step.
4  # Run on a freshly started app: record ids are assumed to be 1.
5  # Open the dumps in a hex viewer and search for the markers.
6  set -e
7  BASE=${BASE:-http://localhost:5080}
8  OUT=${OUT:-./dumps}
9  mkdir -p "$OUT"
10 PID=$(pgrep -f '^dotnet .*SecureApp.Api.dll' | head -1)
11 [ -n "$PID" ] || { echo "SecureApp.Api is not running"; exit 1; }
12
13 curl -s -X POST $BASE/api/auth/register -H 'Content-Type: application/json' \
14     -d '{"username":"dumpuser","password":"DumpUserPass123"}' >/dev/null || true
15 curl -s -c cookies.txt -X POST $BASE/api/auth/login -H 'Content-Type: application/json' \
16     -d '{"username":"dumpuser","password":"DumpUserPass123"}' >/dev/null
17
18 send() { curl -s -b cookies.txt -X "$1" "$BASE/api/memory/$2" -H 'Content-Type:
   ↪ application/json' -d "$3"; }
19
20 echo "1. create"
21 send POST confidential '{"title":"secret","content":"CONFIDENTIAL-MARKER-1"}' >/dev/null
22 send POST public '{"title":"open","content":"PUBLIC-MARKER-1"}' >/dev/null
23 dotnet-dump collect -p $PID -o $OUT/1-create.dmp
24
25 echo "2. update"
26 send PUT confidential/1 '{"title":"secret","content":"CONFIDENTIAL-MARKER-2"}' >/dev/null
27 send PUT public/1 '{"title":"open","content":"PUBLIC-MARKER-2"}' >/dev/null
28 dotnet-dump collect -p $PID -o $OUT/2-update.dmp
29
30 echo "3. delete"
31 send DELETE confidential/1 >/dev/null
32 send DELETE public/1 >/dev/null
33 dotnet-dump collect -p $PID -o $OUT/3-delete.dmp
34
35 rm cookies.txt
36 echo "Marker counts (UTF-8 + UTF-16 strings):"
37 for d in $OUT/*.dmp; do
38     echo "$d: CONFIDENTIAL=$( { strings $d; strings -e l $d; } | grep -c
   ↪ CONFIDENTIAL-MARKER) PUBLIC=$( { strings $d; strings -e l $d; } | grep -c
   ↪ PUBLIC-MARKER)"
39 done
```

Листинг 2 – Скрипт снятия дампов

Число найденных вхождений маркеров (`strings` в UTF-8 и UTF-16) приведено в листинг 3.

```
1 Marker counts (UTF-8 + UTF-16 strings):
2 ./dumps/1-create.dmp: CONFIDENTIAL=5 PUBLIC=4
3 ./dumps/2-update.dmp: CONFIDENTIAL=10 PUBLIC=8
4 ./dumps/3-delete.dmp: CONFIDENTIAL=8 PUBLIC=6
5 3254fa0 {"id":1,"title":"secret","content":"CONFIDENTIAL-MARKER-2","cr
6 32560a6 {"id":1,"title":"secret","content":"CONFIDENTIAL-MARKER-2","cr
7 325616d {"title":"secret","content":"CONFIDENTIAL-MARKER-2"}
8 32595cc {"title":"secret","content":"CONFIDENTIAL-MARKER-1"}
```

Листинг 3 – Вхождения маркеров в дампах и их адреса в дампе после удаления

– Маркеры обоих видов данных есть во всех трёх дампах, в том числе после удаления записи.

– Найденные копии – это тело HTTP-запроса и ответа в формате JSON (в листинге видно строки вида `{"title":"secret","content":"CONFIDENTIAL-MARKER-1"}`), а не содержимое словарей приложения: там лежит только шифротекст.

– Конфиденциальный и открытый маркеры встречаются почти одинаково часто: шифрование защищает данные внутри хранилища, но не буферы, через которые данные приходят по сети и уходят обратно.

Вероятная причина – переиспользуемые буферы, которые `Kestrel` и `System.Text.Json` берут из общего пула и не обнуляют после возврата. Эта гипотеза отдельно не проверялась. Скорее всего, устранить эти копии кодом приложения нельзя.

Выводы

В приложение **SecureApp** добавлен раздел с данными только в оперативной памяти: конфиденциальные записи шифруются AES-256-GCM, неконфиденциальные хранятся открыто. Проведён анализ кода (валидация, работа с секретами, параллельный доступ), измерены память и процессорное время под нагрузкой и сняты дампы памяти после создания, обновления и удаления данных.

Семантически работа показала, что шифрование защищает только данные внутри хранилища приложения. Открытый текст по-прежнему оказывается в памяти процесса из-за буферов обработки запросов, и удаление записи из словаря эти копии не убирает.

Прагматически: чтобы оценить, где остаются секреты, недостаточно прочитать код, нужен дамп памяти. Для данных, которые не должны оставаться в памяти, одного шифрования недостаточно: нужно ограничивать время жизни процесса или использовать среду, где буферами можно управлять.